Exploratory Data Analysis

1). Exploring the Customer Geography:

```
SELECT DISTINCT
    country
FROM gold.dim_customers;
```

Results | Messages

	country
1	n/a
2	Germany
3	United States
4	Australia
5	United Kingdom
6	Canada
7	France

2)- Exploring Customer Demography:-- Retrieve unique gender Composition

```
SELECT DISTINCT
    Gender
FROM gold.dim_customers;
```

95 % No issues found

Results | Messages

	Gender
1	n/a
2	Male
3	Female

3) Product Hierarchy: Explore category → subcategory → product structure

```
SELECT DISTINCT
   category,
   subcategory,
   product_name
FROM gold.dim_products
ORDER BY 1,2,3
```

Results | Messages

	category	subcategory	product_name
1	NULL	NULL	HL Mountain Pedal
2	NULL	NULL	HL Road Pedal
3	NULL	NULL	LL Mountain Pedal
4	NULL	NULL	LL Road Pedal
5	NULL	NULL	ML Mountain Pedal
6	NULL	NULL	ML Road Pedal
7	NULL	NULL	Touring Pedal
8	Accessories	Bike Racks	Hitch Rack - 4-Bike
9	Accessories	Bike Stands	All-Purpose Bike Stand
10	Accessories	Bottles and Cages	Mountain Bottle Cage
11	Accessories	Bottles and Cages	Road Bottle Cage
12	Accessories	Bottles and Cages	Water Bottle - 30 oz.
13	Accessories	Cleaners	Bike Wash - Dissolver
14	Accessories	Fenders	Fender Set - Mountain
15	Accessories	Helmets	Sport-100 Helmet- Black
16	Accessories	Helmets	Sport-100 Helmet- Blue

4). -- Sales Time Range: Identify data coverage period Helps understand how many years of sales data are available

```
SELECT
   MIN(order_date) AS first_order_date,
   MAX(order_date) AS last_order_date,
   DATEDIFF(YEAR, MIN(order_date), MAX(order_date)) AS order_range_years
FROM gold.fact_sales;
```

95 % No issues found

Results Messages

	first_order_date	last_order_date	order_range_years
1	2010-12-29	2014-01-28	4

-5) Customer Age Range: Identify youngest and oldest customer. Age is approximated using year difference

```
SELECT
   MIN(birthdate) AS oldest_birthdate,
   DATEDIFF(YEAR, MIN(birthdate), GETDATE()) AS oldest_age,
   MAX(birthdate) AS youngest_birthdate,
   DATEDIFF(YEAR, MAX(birthdate), GETDATE()) AS youngest_age
FROM gold.dim_customers;
```

95 % No issues found

Results Messages

	oldest_birthdate	oldest_age	youngest_birthdate	youngest_age
1	1916-02-10	110	1986-06-25	40

6.) **- KPI Summary Report (Single Result Set) -- Combines multiple KPIs into one output using UNION ALL**

```
SELECT 'Total_Sales' AS Measure_Name, SUM(sales_amount) AS Measure_Value FROM gold.fact_sales
UNION ALL
SELECT 'Total_Quantity', SUM(quantity) FROM gold.fact_sales
```

```sql
UNION ALL
SELECT 'Avg_Price', AVG(sls_price) FROM gold.fact_sales
UNION ALL
SELECT 'Total_no_Orders', COUNT(DISTINCT order_number) FROM gold.fact_sales
UNION ALL
SELECT 'Total_no_products', COUNT(product_key) FROM gold.dim_products
UNION ALL
SELECT 'Total_no_Customers', COUNT(customer_key) FROM gold.dim_customers;
```

95 % No issues found

Results Messages

	Measure_Name	Measure_Value
1	Total_Sales	29356250
2	Total_Quantity	60423
3	Avg_Price	486
4	Total_no_Orders	27659
5	Total_no_products	295
6	Total_no_Customers	18484

7). -- Customer Distribution by Country

```sql
SELECT
   Country,
   COUNT(Customer_key) AS Total_Customers
FROM gold.dim_customers
GROUP BY Country
ORDER BY Total_Customers DESC;
```

95 % No issues found

Results Messages

	Country	Total_Customers
1	United States	7482
2	Australia	3591
3	United Kingdom	1913
4	France	1810
5	Germany	1780
6	Canada	1571
7	n/a	337

8). Customer Distribution by Gender

```sql
SELECT
    Gender,
    COUNT(Customer_key) AS Total_Customers
FROM gold.dim_customers
GROUP BY Gender
ORDER BY Total_Customers DESC;
```

95 % No issues found

Results Messages

	Gender	Total_Customers
1	Male	9341
2	Female	9128
3	n/a	15

9) - Product Distribution by Category

```sql
SELECT
    Category,
    COUNT(product_key) AS Total_Products
FROM gold.dim_products
GROUP BY Category
ORDER BY Total_Products DESC;
```

Results Messages

	Category	Total_Products
1	Components	127
2	Bikes	97
3	Clothing	35
4	Accessories	29
5	NULL	7

10) -- Average Product Cost by Category

```sql
SELECT
    Category,
    AVG(Cost) AS Avg_Cost
```

```
FROM gold.dim_products
GROUP BY Category
ORDER BY Avg_Cost DESC;
```

Results | Messages

	Category	Avg_Cost
1	Bikes	949
2	Components	264
3	NULL	28
4	Clothing	24
5	Accessories	13

11) -- Revenue by Product Category -- LEFT JOIN ensures all sales are included even if product mapping is missing

```
SELECT
   p.Category,
   SUM(s.sales_amount) AS Total_Revenue
FROM gold.fact_sales AS s
LEFT JOIN gold.dim_products AS p
   ON p.product_key = s.product_key
GROUP BY p.Category
ORDER BY Total_Revenue DESC;
```

Results | Messages

	Category	Total_Revenue
1	Bikes	28316272
2	Accessories	700262
3	Clothing	339716

12) Revenue by Customer

```
SELECT
   c.customer_key,
   c.first_name,
   c.last_name,
   SUM(s.sales_amount) AS Total_Revenue
FROM gold.fact_sales AS s
LEFT JOIN gold.dim_customers AS c
   ON c.customer_key = s.customer_key
GROUP BY
   c.customer_key,
   c.first_name,
   c.last_name
ORDER BY Total_Revenue DESC;
```

Results | Messages

	customer_key	first_name	last_name	Total_Revenue
1	1302	Nichole	Nara	13294
2	1133	Kaitlyn	Henderson	13294
3	1309	Margaret	He	13268
4	1132	Randall	Dominguez	13265
5	1301	Adriana	Gonzalez	13242
6	1322	Rosa	Hu	13215
7	1125	Brandi	Gill	13195
8	1308	Brad	She	13172
9	1297	Francisco	Sara	13164
10	434	Maurice	Shan	12914
11	440	Janet	Munoz	12488
12	242	Lisa	Cai	11468
13	418	Lacey	Zheng	11248
14	421	Jordan	Turner	11200
15	243	Larry	Munoz	11067
16	1656	Larry	Vazquez	10899
17	2264	Kate	Anand	10871
18	1324	Lawrence	Alonso	10836

Query executed successfully.

13) Quantity Sold by Country

```
SELECT
   c.Country,
   SUM(s.quantity) AS Total_quantity
FROM gold.fact_sales AS s
LEFT JOIN gold.dim_customers AS c
   ON c.customer_key = s.customer_key
GROUP BY c.Country
```

```
ORDER BY Total_quantity DESC;
```

Results | Messages

	Country	Total_quantity
1	United States	20481
2	Australia	13346
3	Canada	7630
4	United Kingdom	6910
5	Germany	5626
6	France	5559
7	n/a	871

14) Top 5 Products by Revenue

```
SELECT TOP 5
   p.product_name,
   SUM(s.sales_amount) AS Total_Revenue
FROM gold.fact_sales AS s
LEFT JOIN gold.dim_products AS p
   ON p.product_key = s.product_key
GROUP BY p.product_name
ORDER BY Total_Revenue DESC;
```

Results | Messages

	product_name	Total_Revenue
1	Mountain-200 Black- 46	1373454
2	Mountain-200 Black- 42	1363128
3	Mountain-200 Silver- 38	1339394
4	Mountain-200 Silver- 46	1301029
5	Mountain-200 Black- 38	1294854

15) -- Top Products using Ranking (Flexible approach) : ROW_NUMBER allows dynamic ranking instead of fixed TOP

```
SELECT *
FROM (
   SELECT
      p.product_name,
      SUM(s.sales_amount) AS Total_Revenue,
      ROW_NUMBER() OVER (ORDER BY SUM(s.sales_amount) DESC) AS rank_products
   FROM gold.fact_sales AS s
   LEFT JOIN gold.dim_products AS p
      ON p.product_key = s.product_key
   GROUP BY p.product_name
) t
WHERE rank_products <= 5;
```

95 % No issues found

Results Messages

	product_name	Total_Revenue	rank_products
1	Mountain-200 Black- 46	1373454	1
2	Mountain-200 Black- 42	1363128	2
3	Mountain-200 Silver- 38	1339394	3
4	Mountain-200 Silver- 46	1301029	4
5	Mountain-200 Black- 38	1294854	5

16) Top 10 Customers by Revenue:

```
SELECT TOP 10
   c.customer_key,
   c.first_name,
   c.last_name,
   SUM(s.sales_amount) AS Total_Revenue
FROM gold.fact_sales AS s
LEFT JOIN gold.dim_customers AS c
   ON c.customer_key = s.customer_key
GROUP BY
   c.customer_key,
   c.first_name,
   c.last_name
ORDER BY Total_Revenue DESC;
```

Results | Messages

	customer_key	first_name	last_name	Total_Revenue
1	1302	Nichole	Nara	13294
2	1133	Kaitlyn	Henderson	13294
3	1309	Margaret	He	13268
4	1132	Randall	Dominguez	13265
5	1301	Adriana	Gonzalez	13242
6	1322	Rosa	Hu	13215
7	1125	Brandi	Gill	13195
8	1308	Brad	She	13172
9	1297	Francisco	Sara	13164
10	434	Maurice	Shan	12914

Query executed successfully.

17) Bottom 5 Products by Revenue (Worst Performers)

```
SELECT TOP 5
   p.product_name,
   SUM(s.sales_amount) AS Total_Revenue
FROM gold.fact_sales AS s
LEFT JOIN gold.dim_products AS p
   ON p.product_key = s.product_key
GROUP BY p.product_name
ORDER BY Total_Revenue ASC;
```

Results | Messages

	product_name	Total_Revenue
1	Racing Socks- L	2430
2	Racing Socks- M	2682
3	Patch Kit/8 Patches	6382
4	Bike Wash - Dissolver	7272
5	Touring Tire Tube	7440

Query executed successfully.

18) Customers with Lowest Order Activity:

```
SELECT TOP 3
   c.customer_key,
   c.first_name,
   c.last_name,
   COUNT(DISTINCT s.order_number) AS Total_Order
FROM gold.fact_sales AS s
LEFT JOIN gold.dim_customers AS c
   ON c.customer_key = s.customer_key
GROUP BY
   c.customer_key,
   c.first_name,
   c.last_name
ORDER BY Total_Order ASC;
```

Results | Messages

	customer_key	first_name	last_name	Total_Order
1	16	Chloe	Young	1
2	17	Wyatt	Hill	1
3	21	Jordan	King	1